several crucial words responsible for the calculation logic with irrelevant words. For example, the sentence from X_{r} like "42 plus 34 is equal to _" is replaced to the counterfactual one "42 *nothing* 34 is equal to _". In this way, it allows for a direct reflection of the model's impact on the arithmetic calculation tasks, rather than being influenced by the sentence structure or syntax.

4. Method

Our goal is to interpret the LLMs in a way that is human-understandable, thus enabling targeted modification of models through precise SFT. This section delves into the "identify-analyze-finetune" methodology. First, in Section 4.1, we describe the process of identifying and validating key components within LLMs. Then in Section 4.2, we examine the inherent patterns of these pivotal components to decode their behavior and distinct features. Finally, in Section 4.3, we introduce a strategy of precise SFT that fine-tunes these influential components to enhance the proficiency in calculation.

4.1. Key Components Identification.

The computation of the LLM can be reorganized as a directed acyclic graph (DAG) (Wang et al., 2023a). In the graph, each node is a computation component, including attention heads, MLP layers, residual connections, and each edge represents the data flow that the output of the previous node will be transposed to the input of the later node. Please refer to Appendix B for more details. To unravel the underlying cause of the model's predicted answer, we employ the causal intervention technique known as *path patching* (Goldowsky-Dill et al., 2023; Wang et al., 2023a). By perturbing targeted activation with counterfactual data X_{c} and freezing others with reference data X_{r}, the comparison on output logits is employed to measure the counterfactual effect. The whole process is illustrated in Algorithm 1. In this work, we scan through all nodes $\mathcal{N}$ one by one, and measure the changes in the output logit of ground-truth token $\{\mathtt{C}\}$, recoding in $E_{\mathcal{N}}$. Notably, since the residual operations and MLPs compute each token separately (Elhage et al., 2021), patching the head output at the END position (*i.e.*, the last token in the input sentence) is enough to measure the effects on the next token prediction.

Explanations for model behavior can easily be misleading or non-rigorous (Bolukbasi et al., 2021; Wiegreffe & Pinter, 2019). To address this issue, we further assess the importance of the identified heads/MLPs, while also confirming the insignificance of others. For this purpose, we employ a knockout technique called *mean ablation* (Wang et al., 2023a) to deactivate the individual heads/MLPs and observe their impact on model performance. Specifically, we replace their activation with average activation across counterfactual data X_{c} to remove the task-related information. By observing changes in model performance, we can verify the roles of these key heads/MLPs.

Algorithm 1 Identifying Key Components

Input: Set $\mathbf{\Omega}$ of reference and counterfactual sample pairs $(X_{\mathrm{r}}, X_{\mathrm{c}})$, model $\mathcal{M}$ with nodes $\mathcal{N}$.
Output: Causal effects for $\mathcal{N}$: $E_{\mathcal{N}}$.
for $(X_r^{(i)}, X_c^{(i)})$ in $\mathbf{\Omega}$ **do**
  Compute all activations A_r, A_c on $(X_r^{(i)}, X_c^{(i)})$
  for n in $\mathcal{N}$ **do**
    $A'_r(n) \leftarrow A_c(n)$; ▷ replace output in A_r by A_c
    $A'_r(k) \leftarrow A_r(k), \forall k \in [1, \cdots, |\mathcal{N}|], k \neq n.$
    $logit_o \leftarrow \mathcal{M}(X_r^{(i)}, A_r)$ ▷ get original logits
    $logit_p \leftarrow \mathcal{M}(X_r^{(i)}, A'_r)$ ▷ get patched logits
    $s_n^{(i)} \leftarrow \frac{logit_p - logit_o}{logit_o}$ ▷ causal effect
  end for
end for
Return: $\overline{s_n} = \frac{\sum_{i=1}^{|\mathbf{\Omega}|} s_n^{(i)}}{|\mathbf{\Omega}|}$ ▷ averaged effect w.r.t. samples

4.2. Pattern Analysis.

To make the identified heads/MLPs accessible to human understanding, we conduct a deeper analysis of their operational "behaviors". For attention heads, we examine the attention pattern $A_{i,j} \in \mathbb{R}^{N \times N}$ to comprehend which tokens are prioritized. N is the number of input tokens. Specifically, we begin by gathering the respective attention patterns $A_{i,j}$ on reference data X_r of the key heads. We extract the last row of $A_{i,j}$ for each sample, analyzing the attention scores $A_{i,j}^{END} \in \mathbb{R}^{1 \times N}$ between the Query token at the END position and each Key token, and obtaining the averaged scores w.r.t. samples. Generally, the type of token with the highest attention score represents the characteristics of the head, such as numbers, math symbols, etc.

For MLPs, we use the unembedding matrix as the probing to measure the content of token, especially numerical tokens, contained in MLPs' inputs and outputs. Prior studies, such as those reported in (Elhage et al., 2021), have illustrated that the MLP layer initially receives its input from the residual stream (*i.e.*, MLP_{in}), subsequently adding its output back into that stream (*i.e.*, MLP_{out}). Let W_U represent the unembedding matrix, and $W_U[*]$ denote the unembedding vector corresponding to a specific token. We calculate the cosine similarity between MLP_{in}, MLP_{out} and $W_U[\{\mathtt{A}\}], W_U[\{\mathtt{B}\}], W_U[\{\mathtt{C}\}]$ to reflect the information the MLP receives and generates. To isolate the specific contribution of MLP to specific numerical tokens, we further evaluate the subtraction of outputs and inputs of MLP, *i.e.*, $\frac{MLP_{out} - MLP_{in}}{||MLP_{out} - MLP_{in}||} \cdot \frac{W_U[\{\mathtt{A}\}]}{||W_U[\{\mathtt{A}\}]||}$. Research in (Geva et al., 2022) presents that each MLP layer's output token repre-

sentation can be characterized as an additive update influencing the evolving representation across vocabularies. Our methodology is aligned with these works, while we mainly focus on the token embeddings of right/wrong answers to reveal the contribution of MLPs on the calculation tasks.

4.3. Precise Fine-tuning.

Supervised Fine-Tuning (SFT) is widely used for enhancing a model's mathematical capabilities. Building on this, precise SFT only updates those components closely associated with mathematical abilities, while keeping the rest parameters unchanged. Algorithm 2 illustrates the whole process. For the i-th attention layer, the output matrix W_O^i is split into equal size blocks for each head $\left[W_O^{i,1}, W_O^{i,2}, \cdots W_O^{i,H}\right]$. As

Algorithm 2 Precise Fine-tuning

Require: Model $\mathcal{M}$, input X, index of key heads $\mathbf{\Phi}$, iterations I, learning rate η, $W_\theta = W_{Q/K/V/O}$
for $(i, j) \in \mathbf{\Phi}$ **do**
  $W_\theta^{i,j}.requires_grad = True$
end for ▷ activate key heads
loop I times
  $\mathcal{L}$ = $\mathcal{M}$.forward(X)
  $\mathcal{L}$.backward()
  for $w \in W_\theta$ **do**
    $w = w - \eta * w.grad$
  end for ▷ update target parameters
end loop

is verified in (Elhage et al., 2021), it is equivalent to running heads independently, multiplying each by its own output matrix, and adding them into the residual stream. For the selected individual heads, precise SFT updates the parameters of four matrices: $W_Q^{i,j}$, $W_K^{i,j}$, $W_V^{i,j} \in \mathbb{R}^{d \times \frac{d}{H}}$, and $W_O^{i,j} \in \mathbb{R}^{\frac{d}{H} \times d}$. For the selected MLP layer, precise SFT updates all parameters in this layer. Moreover, since we adjust only a small fraction of the parameters, precise SFT naturally benefits from shorter training times and minimal impact on the model's original capabilities.

5. Experiments

The experiments are organized as follows: (1) *identify* the calculation-related key components via path patching and *validate* their importance in implementing arithmetic calculation via knockout in Section 5.1; (2) *understand* the behavior of the newly identified components by examining their attention patterns and embeddings in Section 5.2; (3) *improve* the mathematical capability via precise supervised fine-tuning on math benchmarks in Section 5.3. For simplicity, we primarily report the results of LLaMA2-7B, while the results of other models can be found in Appendix.

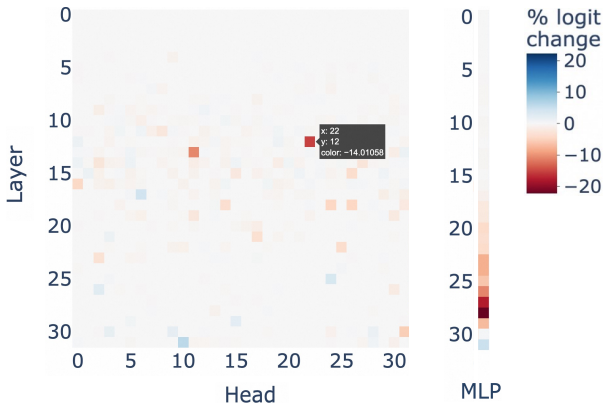


Figure 2: We conduct path patching experiments on LLaMA2-7B across four mathematical tasks, by searching for each head and MLP directly affecting the logit of the right answer. For each head/MLP, a darker color indicates a larger logit difference from the model before patching.

5.1. Identifying Calculation-related Components.

Location of key heads. In Figure 2, we visualize the effect of each head according to the serial numbers of the heads and layers. This arrangement allows for a clear comparison of the causal impact of each head to the logit of ground-truth token {C}. The red squares indicate heads that have a significant positive impact on predicting the output token, while the blue squares represent heads that have a negative effect. From these results, we observe that: (i) *Only a small number of heads have a noteworthy influence on the output*. Specifically, when the heads such as 12.22[3] is patched, there is a substantial decrease of 14.0% on the logit of token {C}, which highlights their positive contribution to the calculation tasks. We classify heads that exhibit logit change exceeding -5% as "key heads". The sparse distribution of these key heads motivates us to explore their specific functionalities and characteristics in Section 5.2. (ii) *The discovered key heads are mainly located in the middle layers*. For LLaMA2-7B, key heads emerge starting from the 12th layer for all arithmetic calculations. Prior layers exhibit heads that do not exert a direct effect on the output logits. Key heads are primarily concentrated between layers 12 and 17. (More analysis of the key heads in other LLMs can be found in Appendix C.)

Location of key MLPs. The last column in Figure 2 visualizes the effect of each MLP layer on the logit of ground-truth token {C}. It is observed that MLPs before the identified heads ($0-16$) have almost no impact on the outputs (approximately $\pm 0.0\%$). In contrast, after the 17-th layer, MLPs exhibit a much larger effect (approximately $\pm 10.0\%$). It indicates that MLPs are engaged in the calculation. We hy-

[3]We apply the notation of $i.j$ to refer to the j-th head of the i-th attention layer.

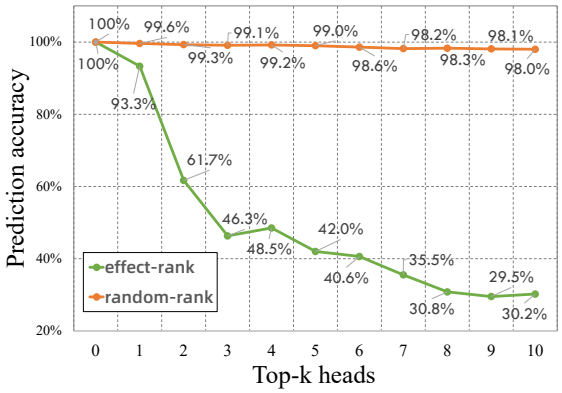


Figure 3: The influence on prediction accuracy after knocking out top-k attention heads that are sorted by the effect of each head on logits ("effect-rank"), and knocking out randomly-sorted top-k heads ("random-rank").

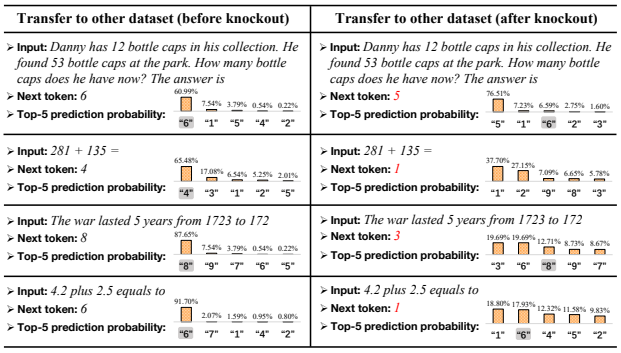
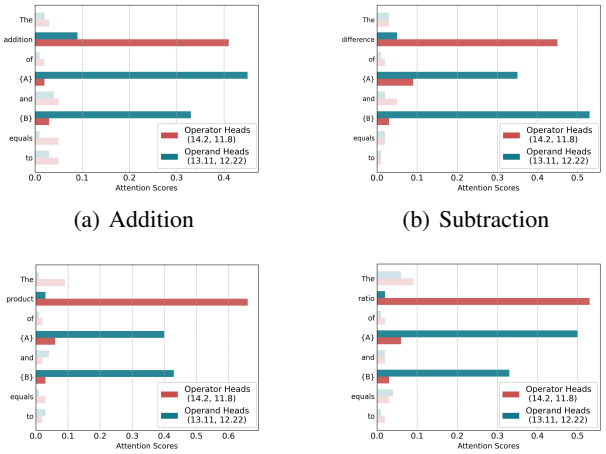


Figure 4: After knocking out the key heads, LLaMA2-7B predicts incorrectly on the cases of SVAMP dataset and other data formats of multi-digit integers, rational numbers.

pothesize that the calculation process is firstly implemented through the key heads, then the subsequent MLPs gradually work out the final results. We validate this in Section 5.2.

Validation of key components. To fully validate the faithfulness of the discovered key heads, we perform additional checks by observing the performance drop when knocking out these components. In Figure 3, all heads are sorted in a certain order by the importance score shown in Fig. 2 and knocked out one by one. It shows that, as the heads are gradually knocked out, the performance of the model drops sharply in "effect-rank", while keeping stable (relatively minor effect within 2%) in "random-rank". We also exhibit the transferability of the key heads with different data prompts or formats as shown in Figure 4. The model becomes largely confused to output incorrect numbers after knocking out the identified key heads. On the dataset SVAMP, there is a relative performance drop ($-22.9\%/34.7\%=-66.0\%$) after the knockout, aligned with the result on our generated dataset. The above results demonstrate that the discovered components play an important role in the language model's ability to complete the calculation task.



(a) Addition (b) Subtraction

(c) Multiplication (d) Division

Figure 5: The attention score distribution of key heads across four calculation tasks. The key heads (*e.g.*, 13.11, 14.2) attend to number operands and calculation operators.

5.2. Understanding Calculation-related Component Behaviors.

Key heads behavior. In order to better understand the "behavior" of the heads that have a significant impact on calculation, we begin by analyzing their attention patterns, and check the attention scores between Query END token and each Key token as illustrated in Sec. 4.2. Our findings reveal that these heads exhibit a strong focus on tokens of operands or operators. For example, heads 13.11 and 12.22 have high attention scores on numbers including {A} and {B}, while heads 14.2 and 11.8 attend more to symbols or text indicating operations like "+", "-", "plus", "div", etc. We randomly select 1000 samples from reference data and plot the distribution of averaged attention scores on key heads (arranged in two groups) for four arithmetic calculations.

As illustrated in Figure 5, the operand heads and the operator heads are colored in red and green respectively, and highlighted at the positions of operands and operators. It is clear that these heads exhibit distinctly different distributions and show minimal attention to tokens outside of the operands/operators. Moreover, we visualize the attention patterns of the key heads (*e.g.*, 13.11) on various types of sentences in Figure 7. It reveals that the key heads also primarily prioritize number operands (*e.g.*, '1' and '5' in the first case) even for unseen data formats. This observation provides an explanation for why the deactivation of the key heads can influence the model's perception on number tokens and consequently affect its prediction when transfer-